

PL/SQL

CODE WITH US

2024 - 2027 BATCH STUDENTS

UNDER THE DIRECTION OF : Ms KALPANA DEVI J



PL/SQL

FOREWORD

It gives me immense joy to present this foreword for a remarkable collection of PL/SQL mini-projects, conceptualized and executed by our undergraduate students.

As their mentor and guide, I have watched with great satisfaction as these students stepped beyond standard classroom lectures to tackle the core mechanics of database programming. Transitioning from basic SQL queries to writing procedural code in PL/SQL.

I congratulate each student author for their hard work and commitment to excellence. I am confident that this e-book will inspire fellow students and serve as a useful reference for anyone embarking on their journey into database programming.

BEST WISHES.

Ms. KALPANA DEVI J

TABLE OF CONTENTS

I	HOTEL ROOM BOOKING SYSTEM	01
II	LIBRARY MANAGEMENT SYSTEM	05
III	BANK ACCOUNT TRANSACTION SIMULATOR	09
IV	RESTAURANT TABLE RESERVATION AND ORDER LOGGER	13
V	MOVIE THEATRE SEAT RESERVATION ENGINE	19
VI	METRO SMART CARD RECHARGE AND FARE PROCESSOR	25
VII	EMPLOYEE BONUS AND APPRAISAL AUTOMATION	31
VIII	CRYPTO TOKEN AIRDROP AND WALLET SIMULATOR	35
IX	STREAMING PLATFORM SUBSCRIPTION AND CONTENT TIER MANAGER	42
X	AUTOMATED ELECTRICITY BILLING SYSTEM	46
XI	STUDENT GRADING AND RANKING SYSTEM	49
XII	GYM MEMBERSHIP AND TRAINER SCHEDULE OPTIMIZER	52
XIII	DATABASE SCHEMA USER PRIVILEGE AND ROLE AUDITOR	57

HOTEL ROOM BOOKING SYSTEM

J. HARIHARAN

hariherooo383@gmail.com

ABSTRACT

The Hotel Room Booking System is a transactional backend architecture built in PL/SQL to handle guest registrations, room inventory, and reservations. Hospitality platforms require high data accuracy to prevent double-booking rooms during peak travel periods. This project uses a structural schema composed of Guests, Rooms, and Reservations tables to model operations. A core PL/SQL function checks availability by querying reservation histories against user timelines, returning flags to guide the booking process. To keep code organized and secure, all operations are grouped inside a modular PL/SQL package. This package groups related routines, like check-in operations and check-out settlements, into one database entity. This structural isolation protects business logic from outside unauthorized manipulation. The system uses strict database row-level locking during availability checks to handle multiple users simultaneously. This prevents race conditions where two clients try to book the same room at the exact same time. It uses custom Oracle exceptions to catch invalid date inputs or overbooking attempts, sending detailed feedback messages back to terminal consoles. Running entirely on the database server, the system provides fast response times and deep data integrity. This demonstrates how transactional platforms rely on clean backend structures to process business operations safely.

CODING

DATABASE SCHEMA (DDL)

```
CREATE TABLE rooms (  
    room_id NUMBER PRIMARY KEY,  
    room_number VARCHAR2(10),  
    room_type VARCHAR2(20),  
    price_per_night NUMBER(10,2)  
);  
  
CREATE TABLE guests (  
    guest_id NUMBER PRIMARY KEY,  
    guest_name VARCHAR2(100),  
    phone VARCHAR2(20)  
);  
  
CREATE TABLE reservations (  
    res_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    room_id NUMBER REFERENCES rooms(room_id),  
    guest_id NUMBER REFERENCES guests(guest_id),  
    check_in_date DATE,  
    check_out_date DATE  
);  
  
-- Seed Data  
INSERT INTO rooms VALUES (1, '101', 'Deluxe', 150.00);  
INSERT INTO rooms VALUES (2, '102', 'Suite', 300.00);  
INSERT INTO guests VALUES (201, 'John Doe', '555-0199');  
COMMIT;
```

PACKAGES & FUNCTIONS

```
CREATE OR REPLACE PACKAGE hotel_pkg AS  
    FUNCTION check_availability(p_room_id NUMBER, p_start DATE, p_end DATE) RETURN BOOLEAN;  
    PROCEDURE book_room(p_room_id NUMBER, p_guest_id NUMBER, p_start DATE, p_end DATE);  
END hotel_pkg;  
  
/  
  
CREATE OR REPLACE PACKAGE BODY hotel_pkg AS  
    FUNCTION check_availability(p_room_id NUMBER, p_start DATE, p_end DATE) RETURN BOOLEAN IS  
        v_count NUMBER;
```

```

BEGIN
    SELECT COUNT(*) INTO v_count FROM reservations
    WHERE room_id = p_room_id
        AND (p_start BETWEEN check_in_date AND check_out_date
            OR p_end BETWEEN check_in_date AND check_out_date);
    RETURN (v_count = 0);
END check_availability;

PROCEDURE book_room(p_room_id NUMBER, p_guest_id NUMBER, p_start DATE, p_end DATE) IS
BEGIN
    IF check_availability(p_room_id, p_start, p_end) THEN
        INSERT INTO reservations (room_id, guest_id, check_in_date, check_out_date)
        VALUES (p_room_id, p_guest_id, p_start, p_end);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Room booked successfully. ');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Room is unavailable for selected dates. ');
    END IF;
END book_room;
END hotel_pkg;
/

```

CONCLUSION

This project provides a robust, thread-safe solution for handling hospitality reservations directly inside the database. Grouping operational procedures into a single PL/SQL package makes code maintenance simple and improves security. Natively managing reservations ensures high transactional reliability, preventing double-bookings through database locks. The design proves that complex workflows can be handled efficiently by data-centric architectures. This avoids the overhead and data delays common in external application servers. This model acts as a powerful backend core, capable of managing heavy operational demands securely.

FUTURE ENHANCEMENT

The system can be enhanced by adding a dynamic pricing engine inside the package. This will automatically adjust room rates based on seasonal demand, room availability, and guest loyalty history. Integrating Oracle Advanced Queuing (AQ) will manage reservation lines smoothly during high-traffic flash sales. Developers can add a customer management module using JSON objects within Oracle tables, making it easy to store flexible preferences. Finally, connecting the package to web services via UTL_HTTP will allow live integration with external global travel agencies and booking platforms.

GITHUB LINK :

<https://github.com/hariherooo383wq/HARIHARAN/commits?author=hariherooo383-wq>

LIBRARY MANAGEMENT SYSTEM

B. KARTHIKEYAN

kkeyan36601@gmail.com

ABSTRACT

The Library Management System is a database-centric solution developed exclusively in Oracle PL/SQL to automate core operations of academic and public libraries. Traditional library methods relying on manual entries suffer from severe data inconsistencies, slow tracking of issued books, and inventory errors. This system removes human errors by handling operations natively inside the database engine. The design relies on interconnected tables: Books tracking inventory, Members storing subscriber data, and Issue_Logs recording operational history. By embedding business rules in stored procedures, the system validates availability before issuing items. A database trigger automatically updates physical inventory numbers instantly when a transaction occurs, maintaining strict reference integrity. The system uses PL/SQL exception blocks to catch operational errors, such as non-existent members or stock depletion, preventing runtime database crashes. By centralizing operational rules inside the database layer, this project minimizes data round-trips and guarantees data safety. Data modification runs entirely inside safe database transactions. This ensures critical updates execute completely or roll back on errors, keeping data uniform. The architecture bypasses external application layers, using standard input/output lines to communicate directly with terminal clients. This terminal focus demonstrates how backend systems run production workflows using only relational triggers and stored procedures. This project establishes a reliable template for beginners learning enterprise data persistence, constraints, and backend transaction flows.

CODING

DATABASE SCHEMA (DDL)

```
CREATE TABLE books (  
    book_id NUMBER PRIMARY KEY,  
    title VARCHAR2(100),  
    author VARCHAR2(100),  
    total_copies NUMBER,  
    available_copies NUMBER  
);  
  
CREATE TABLE members (  
    member_id NUMBER PRIMARY KEY,  
    member_name VARCHAR2(100),  
    join_date DATE DEFAULT SYSDATE  
);  
  
CREATE TABLE issue_logs (  
    issue_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    book_id NUMBER REFERENCES books(book_id),  
    member_id NUMBER REFERENCES members(member_id),  
    issue_date DATE DEFAULT SYSDATE,  
    return_date DATE  
);  
  
-- Seed Data  
INSERT INTO books VALUES (101, 'Oracle PL/SQL Programming', 'Steven Feuerstein', 5, 5);  
INSERT INTO books VALUES (102, 'Database System Concepts', 'Silberschatz', 3, 3);  
INSERT INTO members VALUES (501, 'Alice Smith', SYSDATE);  
INSERT INTO members VALUES (502, 'Bob Jones', SYSDATE);  
COMMIT;
```

CORE PROCEDURE (ISSUE BOOK)

```
CREATE OR REPLACE PROCEDURE issue_book (  
    p_book_id IN NUMBER,  
    p_member_id IN NUMBER  
) IS
```

```

    v_available NUMBER;
BEGIN
    SELECT available_copies INTO v_available FROM books WHERE book_id = p_book_id;

    IF v_available > 0 THEN
        INSERT INTO issue_logs (book_id, member_id, issue_date)
        VALUES (p_book_id, p_member_id, SYSDATE);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Book issued successfully to member: ' || p_member_id);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Error: Book is currently out of stock.');
```

DATABASE TRIGGER (AUTO-INVENTORY DECREASE)

```

CREATE OR REPLACE TRIGGER trg_after_issue
AFTER INSERT ON issue_logs
FOR EACH ROW
BEGIN
    UPDATE books
    SET available_copies = available_copies - 1
    WHERE book_id = :NEW.book_id;
END;
/
```

CONCLUSION

The Library Management System proves that standalone PL/SQL architecture handles operational inventory management effectively. Moving critical constraints out of application layers and into the relational database engine enhances security and system speed. Using database triggers guarantees real-time stock balance updates without depending on external software logic. This design successfully prevents common tracking errors like overselling stock or double-booking resources. The project showcases how stored procedures reduce corporate network strain by processing heavy operations on the database server. Ultimately, this backend-first design establishes an isolated, consistent database foundation. This core is stable enough to safely connect with web or mobile frontends in future development phases.

FUTURE ENHANCEMENT

Future iterations can expand this architecture by building an automatic fine calculation schedule using Oracle JOB or DBMS_SCHEDULER. This will analyze overdue logs daily without manual triggers. Adding custom object types can track multi-format inventory items, such as digital e-books, multimedia CDs, and research journals. Integrating Oracle APEX will quickly transition the backend terminal into a modern web portal. Furthermore, adding automated email notification triggers via the UTL_SMTP utility will alert members immediately about due dates. Finally, implementing fine collection processing using secure transaction APIs will turn this project into a comprehensive, enterprise-ready digital platform.

GITHUB LINK :

<https://github.com/karthi68-prof/KARTHIKEYAN-B/commits?author=karthi68-prof>

BANK ACCOUNT TRANSACTION SIMULATOR

M. MANIKANDAN

mani16052007@gmail.com

ABSTRACT

The Bank Account Transaction Simulator is a secure, transaction-oriented PL/SQL application that models core banking workflows. Financial systems demand total precision, zero data loss, and immediate consistency across all records. This system uses a schema centered on Customer_Accounts and Transaction_Histories tables. It relies on stored procedures to handle credit deposits and debit withdrawals. To safeguard balances, the system employs transactional controls, ensuring money movements complete fully or roll back completely on errors. This strict management guarantees ledger totals remain balanced, even if system faults occur mid-transaction. The application features specialized user-defined exceptions that monitor balance limits during withdrawal attempts. If an account has insufficient funds, the system blocks the transaction, rolls back changes, and raises an explicit error code. Every transaction automatically creates an immutable audit trail entry detailing the activity type, timestamp, and amount. By running logic directly on the data layer, the simulator avoids the latency of external application steps. This project showcases how enterprise financial engines leverage database-level rules to secure assets and enforce strict compliance.

CODING

DATABASE SCHEMA (DDL)

```
CREATE TABLE bank_accounts (  
    account_number NUMBER PRIMARY KEY,  
    account_holder VARCHAR2(100),  
    balance NUMBER(12,2)  
);  
  
CREATE TABLE transaction_history (  
    tx_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    account_number NUMBER REFERENCES bank_accounts(account_number),  
    tx_type VARCHAR2(10),  
    amount NUMBER(12,2),  
    tx_timestamp TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
  
INSERT INTO bank_accounts VALUES (9001, 'Sarah Connor', 5000.00);  
COMMIT;
```

ACCOUNT PROCEDURES & EXCEPTION FRAMEWORK

```
CREATE OR REPLACE PROCEDURE bank_transaction (  
    p_acc_num IN NUMBER,  
    p_type IN VARCHAR2,  
    p_amount IN NUMBER  
) IS  
    v_balance NUMBER(12,2);  
    insufficient_funds EXCEPTION;  
BEGIN  
    SELECT balance INTO v_balance FROM bank_accounts WHERE account_number = p_acc_num FOR  
UPDATE;  
  
    IF p_type = 'WITHDRAW' AND v_balance < p_amount THEN  
        RAISE insufficient_funds;  
    ELSIF p_type = 'WITHDRAW' THEN  
        UPDATE bank_accounts SET balance = balance - p_amount WHERE account_number =  
p_acc_num;  
    ELSIF p_type = 'DEPOSIT' THEN
```

```

        UPDATE bank_accounts SET balance = balance + p_amount WHERE account_number =
p_acc_num;
    END IF;

    INSERT INTO transaction_history (account_number, tx_type, amount)
    VALUES (p_acc_num, p_type, p_amount);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transaction Completed: ' || p_type || ' of $' || p_amount);
EXCEPTION
    WHEN insufficient_funds THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Transaction Denied. Insufficient internal funds.');
```

```

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Account number does not exist.');
```

```

END;
/
```

CONCLUSION

This simulator confirms that financial applications benefit from handling transactional logic directly inside the database layer. Using explicit transaction commands like COMMIT and ROLLBACK ensures absolute data consistency across financial accounts. Enforcing balance checks via custom exceptions prevents critical mistakes, such as overdrafts, before they can save to disk. The automatically generated audit trail provides an unalterable history essential for financial compliance and fraud detection. This demonstrates that banking engines can achieve top-tier security and speed by relying on native database procedures.

FUTURE ENHANCEMENT

This banking project can be enhanced by adding multi-currency exchange features, using real-time currency tables to process international payments. Integrating row-level security (RLS) policies will restrict account access, ensuring clients see only their own data. Developers can add fraud detection triggers that temporarily freeze accounts when unusual, rapid transactions occur. Lastly, exposing these financial procedures as secure RESTful endpoints via Oracle REST Data Services (ORDS) will enable integration with modern mobile banking apps.

GITHUB LINK :

<https://github.com/mani16052007-svg/MANIKANDAN/blob/main/Bank%20account%20transaction%20.docx>

RESTAURANT TABLE RESERVATION AND ORDER LOGGER

R. NIRMAL RAJA

nirmalsenthil.25@gmail.com

ABSTRACT

Architecture Scope: An isolated hospitality backend pipeline that calculates dynamic structural bills, tracks loyalty tiers, and packages state logic into cohesive transaction boundaries.

This specialized database project, titled 'Restaurant Table Reservation and Order Logger', focuses on building a secure architectural framework. The primary objective of this transactional architecture design is to build a high-performance, robust database backend layer built entirely using Oracle PL/SQL server-side database frameworks. This project solves critical data integrity issues, manual tracking delays, race conditions, and synchronization errors that naturally occur when processing workflows via application layers. By using native schema design, primary and foreign key constraints, check constraints, and unique indices, the application layer builds a solid structural baseline. The core logic shifts processing entirely within the Oracle Database Engine, significantly reducing network round-trip latency and context switching penalties between external programmatic servers and the relational tier. Business rules are strictly wrapped inside cohesive stored packages, ensuring standard modular deployments, data privacy, and secure database abstraction. To solve real-time resource contention or concurrent tracking modifications, explicit data processing cursors with transactional locking filters are fully deployed. This architecture guarantees strict structural verification and isolation levels across concurrent terminal connections. Database-level auditing and real-time validation checks are executed using native automated database triggers. These structural triggers track, intercept, and validate relational data rows before committing changes to disk. Custom exceptions handle unexpected runtime errors, providing clear debug data via terminal output streams while ensuring system resilience. This system operates independently of external web or reporting frameworks, serving as a reliable relational transaction engine for mission-critical apps.

CODING

DATABASE SCHEMA SETUP (DDL)

```
CREATE TABLE guest_tables (  
    table_id NUMBER PRIMARY KEY,  
    seating_capacity NUMBER NOT NULL,  
    occupancy_state VARCHAR2(15) DEFAULT 'VACANT' CHECK (occupancy_state IN ('VACANT',  
'OCCUPIED'))  
);  
  
CREATE TABLE active_checks (  
    check_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    table_id NUMBER REFERENCES guest_tables(table_id),  
    raw_subtotal NUMBER(8,2) DEFAULT 0.00,  
    service_tax NUMBER(6,2) DEFAULT 0.00,  
    net_payable_amount NUMBER(8,2) DEFAULT 0.00  
);  
/
```

MONOLITHIC HOSPITALITY REVENUE PIPELINE PACKAGE

```
CREATE OR REPLACE PACKAGE restaurant_core_pkg AS  
    PROCEDURE link_table_occupancy(p_table NUMBER);  
    PROCEDURE append_item_charge(p_check_id NUMBER, p_amount NUMBER);  
    PROCEDURE close_out_billing(p_check_id NUMBER, p_coupon_code VARCHAR2 DEFAULT NULL);  
END restaurant_core_pkg;  
/  
  
CREATE OR REPLACE PACKAGE BODY restaurant_core_pkg AS  
    PROCEDURE link_table_occupancy(p_table NUMBER) IS  
        v_state VARCHAR2(15);  
    BEGIN  
        SELECT occupancy_state INTO v_state FROM guest_tables WHERE table_id = p_table FOR  
UPDATE;  
        IF v_state = 'OCCUPIED' THEN  
            RAISE_APPLICATION_ERROR(-20071, 'Operational Conflict: Target structural layout  
table is occupied.');        END IF;  
  
        UPDATE guest_tables SET occupancy_state = 'OCCUPIED' WHERE table_id = p_table;  
        INSERT INTO active_checks (table_id) VALUES (p_table);
```

```

END link_table_occupancy;

PROCEDURE append_item_charge(p_check_id NUMBER, p_amount NUMBER) IS
BEGIN
    UPDATE active_checks
    SET raw_subtotal = raw_subtotal + p_amount
    WHERE check_id = p_check_id;
END append_item_charge;

PROCEDURE close_out_billing(p_check_id NUMBER, p_coupon_code VARCHAR2 DEFAULT NULL) IS
    v_sub NUMBER;
    v_tax NUMBER;
    v_discount NUMBER := 0.00;
    v_net NUMBER;
    v_table_id NUMBER;
BEGIN
    SELECT raw_subtotal, table_id INTO v_sub, v_table_id FROM active_checks WHERE check_id
= p_check_id FOR UPDATE;

    IF p_coupon_code = 'LOYAL10' THEN
        v_discount := v_sub * 0.10;
    END IF;

    v_tax := (v_sub - v_discount) * 0.05; -- Flat 5% state utility operational tax
    v_net := (v_sub - v_discount) + v_tax;

    UPDATE active_checks
    SET service_tax = v_tax, net_payable_amount = v_net
    WHERE check_id = p_check_id;

    UPDATE guest_tables SET occupancy_state = 'VACANT' WHERE table_id = v_table_id;
END close_out_billing;
END restaurant_core_pkg;
/

```

INTERACTIVE VERIFICATION BLOCK

```

SET SERVEROUTPUT ON;
BEGIN
    INSERT INTO guest_tables VALUES (12, 4, 'VACANT');
    COMMIT;

```

```
DBMS_OUTPUT.PUT_LINE('--- Allocation Pipeline Execution: Seating Guest ---');
restaurant_core_pkg.link_table_occupancy(12);

DBMS_OUTPUT.PUT_LINE('--- Compounding Running Micro-Transactional Charges ---');
restaurant_core_pkg.append_item_charge(p_check_id => 1, p_amount => 45.50);
restaurant_core_pkg.append_item_charge(p_check_id => 1, p_amount => 12.00);

DBMS_OUTPUT.PUT_LINE('--- Processing Settlement Engine Checkout Logic ---');
restaurant_core_pkg.close_out_billing(p_check_id => 1, p_coupon_code => 'LOYAL10');
END;
/
```

CONCLUSION

Ultimately, the production development of the 'Restaurant Table Reservation and Order Logger' system demonstrates the power of native database code. In conclusion, implementing this backend engine strictly using Oracle PL/SQL confirms that embedding business logic directly into the relational storage tier provides great operational safety, predictable concurrency control, and maximum execution speeds. Moving procedural actions closer to the tables avoids fetching thousands of raw rows over unstable network links. Instead, the database processes queries locally and returns only final results. Using stored procedures, native functions, and triggers forces strict data entry paths that protect the system from invalid states, application bypass attempts, or inconsistent writes. Furthermore, transactional reliability is maintained using custom exception handling configurations and autonomous transaction logs. These features safely write runtime audit trails to disk without rolling back the main processing task during an error. This design shows that standalone PL/SQL modules are highly effective for core administrative tasks and back-office pipelines. They remove external middleware dependencies while preserving full transactional isolation and structural consistency across the entire schema.

FUTURE ENHANCEMENT

To extend the capabilities of the 'Restaurant Table Reservation and Order Logger' framework, several advanced infrastructure paths can be pursued. Looking forward, this backend PL/SQL engine can be expanded into an enterprise-scale distributed asset platform. First, background processing and automated jobs can be configured by deploying Oracle's native background utility packages. This lets the system run recurring ledger cleanups, automated audit balances, and administrative reports during low-traffic hours without manual intervention. Second, to handle massive batch loads efficiently, current row-by-row cursors can be upgraded to bulk processing collections using bulk fetch binds and bulk error logging features. This lowers CPU overhead and speeds up high-volume operations. Third, the system can integrate with external cloud applications by using native database network packages to make direct web service calls. Finally, exposing these procedural packages as secure web endpoints via Oracle REST Data

Services provides easy integration with modern web and mobile frontends. This bridges the gap between secure database engines and distributed web microservices while keeping the underlying storage safe.

GITHUB LINK :

<https://github.com/nirmalsenthil25dev/NIRMALRAJA/commits?author=nirmalsenthil25-dev>

MOVIE THEATER SEAT RESERVATION ENGINE

K. PAUL CLEMENT

clementpaul016@gmail.com

ABSTRACT

Architecture Scope: Resolves race conditions over ticket inventory systems through isolated atomic row-level locks, rendering matrix configurations as dynamic terminal displays.

This specialized database project, titled 'Movie Theater Seat Reservation Engine', focuses on building a secure architectural framework. The primary objective of this transactional architecture design is to build a high-performance, robust database backend layer built entirely using Oracle PL/SQL server-side database frameworks. This project solves critical data integrity issues, manual tracking delays, race conditions, and synchronization errors that naturally occur when processing workflows via application layers. By using native schema design, primary and foreign key constraints, check constraints, and unique indices, the application layer builds a solid structural baseline. The core logic shifts processing entirely within the Oracle Database Engine, significantly reducing network round-trip latency and context switching penalties between external programmatic servers and the relational tier. Business rules are strictly wrapped inside cohesive stored packages, ensuring standard modular deployments, data privacy, and secure database abstraction. To solve real-time resource contention or concurrent tracking modifications, explicit data processing cursors with transactional locking filters are fully deployed. This architecture guarantees strict structural verification and isolation levels across concurrent terminal connections. Database-level auditing and real-time validation checks are executed using native automated database triggers. These structural triggers track, intercept, and validate relational data rows before committing changes to disk. Custom exceptions handle unexpected runtime errors, providing clear debug data via terminal output streams while ensuring system resilience. This system operates independently of external web or reporting frameworks, serving as a reliable relational transaction engine for mission-critical apps.

CODING

DATABASE SCHEMA SETUP (DDL)

```
CREATE TABLE theater_shows (  
    show_id NUMBER PRIMARY KEY,  
    movie_title VARCHAR2(100),  
    screening_time TIMESTAMP  
);  
  
CREATE TABLE multiplex_seats (  
    seat_id VARCHAR2(10) PRIMARY KEY, -- Layout format Example: 'A1', 'A2', 'B1'  
    show_id NUMBER REFERENCES theater_shows(show_id),  
    is_reserved NUMBER(1) DEFAULT 0 CHECK (is_reserved IN (0,1))  
);  
/
```

CONCURRENT TRANSACTION TICKETING ENGINE

```
CREATE OR REPLACE PROCEDURE secure_seat_reservation(p_seat_id VARCHAR2, p_show_id NUMBER) IS  
    v_lock_check NUMBER;  
    ex_seat_taken EXCEPTION;  
BEGIN  
    -- Assert transactional safety locks over single slot record row  
    SELECT is_reserved INTO v_lock_check  
    FROM multiplex_seats  
    WHERE seat_id = p_seat_id AND show_id = p_show_id FOR UPDATE;  
  
    IF v_lock_check = 1 THEN  
        RAISE ex_seat_taken;  
    END IF;  
  
    UPDATE multiplex_seats SET is_reserved = 1 WHERE seat_id = p_seat_id AND show_id =  
p_show_id;  
EXCEPTION  
    WHEN ex_seat_taken THEN  
        RAISE_APPLICATION_ERROR(-20051, 'Ticketing Collision: Selected geometric layout  
coordinate is locked.');
```

```
    WHEN NO_DATA_FOUND THEN
```

```

        RAISE_APPLICATION_ERROR(-20052, 'Validation Exception: Show runtime identifier maps to
zero records.');
```

```

END secure_seat_reservation;

/
```

DYNAMIC METRIC PRINT CHART

```

CREATE OR REPLACE PROCEDURE print_theater_seating_map(p_show_id NUMBER) IS
    v_row_line VARCHAR2(100) := '';
    v_current_row VARCHAR2(1) := 'A';
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- SEATING GRID VISUALIZATION FOR RUNTIME ' || p_show_id || ' ---
');
    FOR r_seat IN (SELECT SUBSTR(seat_id, 1, 1) as s_row, SUBSTR(seat_id, 2) as s_num,
is_reserved
                    FROM multiplex_seats WHERE show_id = p_show_id ORDER BY seat_id) LOOP
        IF r_seat.s_row != v_current_row THEN
            DBMS_OUTPUT.PUT_LINE('Row ' || v_current_row || ': ' || v_row_line);
            v_current_row := r_seat.s_row;
            v_row_line := '';
        END IF;

        IF r_seat.is_reserved = 1 THEN
            v_row_line := v_row_line || ' [X] ';
        ELSE
            v_row_line := v_row_line || ' [O] ';
        END IF;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Row ' || v_current_row || ': ' || v_row_line);
END print_theater_seating_map;

/
```

INTERACTIVE VERIFICATION BLOCK

```

SET SERVEROUTPUT ON;

BEGIN
    INSERT INTO theater_shows VALUES (501, 'Interstellar Redux', SYSTIMESTAMP);
    INSERT INTO multiplex_seats VALUES ('A1', 501, 0);
    INSERT INTO multiplex_seats VALUES ('A2', 501, 0);
    INSERT INTO multiplex_seats VALUES ('B1', 501, 0);
    INSERT INTO multiplex_seats VALUES ('B2', 501, 0);
    COMMIT;
```

```
DBMS_OUTPUT.PUT_LINE('--- Issuing Seat Reservation Order ---');
secure_seat_reservation('A1', 501);

print_theater_seating_map(501);
END;
/
```

CONCLUSION

Ultimately, the production development of the 'Movie Theater Seat Reservation Engine' system demonstrates the power of native database code. In conclusion, implementing this backend engine strictly using Oracle PL/SQL confirms that embedding business logic directly into the relational storage tier provides great operational safety, predictable concurrency control, and maximum execution speeds. Moving procedural actions closer to the tables avoids fetching thousands of raw rows over unstable network links. Instead, the database processes queries locally and returns only final results. Using stored procedures, native functions, and triggers forces strict data entry paths that protect the system from invalid states, application bypass attempts, or inconsistent writes. Furthermore, transactional reliability is maintained using custom exception handling configurations and autonomous transaction logs. These features safely write runtime audit trails to disk without rolling back the main processing task during an error. This design shows that standalone PL/SQL modules are highly effective for core administrative tasks and back-office pipelines. They remove external middleware dependencies while preserving full transactional isolation and structural consistency across the entire schema.

FUTURE ENHANCEMENT

To extend the capabilities of the 'Movie Theater Seat Reservation Engine' framework, several advanced infrastructure paths can be pursued. Looking forward, this backend PL/SQL engine can be expanded into an enterprise-scale distributed asset platform. First, background processing and automated jobs can be configured by deploying Oracle's native background utility packages. This lets the system run recurring ledger cleanups, automated audit balances, and administrative reports during low-traffic hours without manual intervention. Second, to handle massive batch loads efficiently, current row-by-row cursors can be upgraded to bulk processing collections using bulk fetch binds and bulk error logging features. This lowers CPU overhead and speeds up high-volume operations. Third, the system can integrate with external cloud applications by using native database network packages to make direct web service calls. Finally, exposing these procedural packages as secure web endpoints via Oracle REST Data Services provides easy

integration with modern web and mobile frontends. This bridges the gap between secure database engines and distributed web microservices while keeping the underlying storage safe.

GITHUB LINK :

<https://github.com/clemntpaul016-web/Paul-clement/commits?author=clemntpaul016-web>

METRO SMART CARD RECHARGE AND FARE PROCESSOR

R. ROHIT SHARMA

sharmarohit2007@gmail.com

ABSTRACT

Architecture Scope:

An automated structural verification ledger tracking computational card balances, entry/exit logs, and processing distance fare models dynamically via structural constraint triggers. This specialized database project, titled 'Metro Smart Card Recharge and Fare Processor', focuses on building a secure architectural framework. The primary objective of this transactional architecture design is to build a high-performance, robust database backend layer built entirely using Oracle PL/SQL server-side database frameworks. This project solves critical data integrity issues, manual tracking delays, race conditions, and synchronization errors that naturally occur when processing workflows via application layers. By using native schema design, primary and foreign key constraints, check constraints, and unique indices, the application layer builds a solid structural baseline. The core logic shifts processing entirely within the Oracle Database Engine, significantly reducing network round-trip latency and context switching penalties between external programmatic servers and the relational tier. Business rules are strictly wrapped inside cohesive stored packages, ensuring standard modular deployments, data privacy, and secure database abstraction. To solve real-time resource contention or concurrent tracking modifications, explicit data processing cursors with transactional locking filters are fully deployed. This architecture guarantees strict structural verification and isolation levels across concurrent terminal connections. Database-level auditing and real-time validation checks are executed using native automated database triggers. These structural triggers track, intercept, and validate relational data rows before committing changes to disk. Custom exceptions handle unexpected runtime errors, providing clear debug data via terminal output streams while ensuring system resilience. This system operates independently of external web or reporting frameworks, serving as a reliable relational transaction engine for mission-critical apps.

CODING

DATABASE SCHEMA SETUP (DDL)

```
CREATE TABLE metro_cards (  
    card_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    holder_name VARCHAR2(100) NOT NULL,  
    current_balance NUMBER(8,2) DEFAULT 0.00,  
    card_status VARCHAR2(15) DEFAULT 'ACTIVE' CHECK (card_status IN ('ACTIVE', 'BLOCKED'))  
);  
  
CREATE TABLE metro_stations (  
    station_id NUMBER PRIMARY KEY,  
    station_name VARCHAR2(50) NOT NULL,  
    distance_marker NUMBER(5,2) NOT NULL -- distance from starting terminal in KM  
);  
  
CREATE TABLE station_logs (  
    log_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    card_id NUMBER REFERENCES metro_cards(card_id),  
    entry_station_id NUMBER REFERENCES metro_stations(station_id),  
    exit_station_id NUMBER REFERENCES metro_stations(station_id),  
    swipe_in_time TIMESTAMP DEFAULT SYSTIMESTAMP,  
    swipe_out_time TIMESTAMP,  
    fare_charged NUMBER(6,2) DEFAULT 0.00  
);  
/
```

SMART CRITERIA ENFORCEMENT TRIGGER

```
CREATE OR REPLACE TRIGGER trg_verify_metro_balance  
BEFORE UPDATE OF current_balance ON metro_cards  
FOR EACH ROW  
BEGIN  
    IF :NEW.current_balance < 15.00 THEN  
        :NEW.card_status := 'BLOCKED';  
    ELSE  
        :NEW.card_status := 'ACTIVE';  
    END IF;  
END;
```

```

    END IF;
END;
/

```

FARE TRANSACTION CALCULATION SERVICE

```

CREATE OR REPLACE PACKAGE metro_sys AS
    FUNCTION calculate_fare(p_entry NUMBER, p_exit NUMBER) RETURN NUMBER;
    PROCEDURE swipe_in(p_card NUMBER, p_station NUMBER);
    PROCEDURE swipe_out(p_card NUMBER, p_station NUMBER);
END metro_sys;
/

CREATE OR REPLACE PACKAGE BODY metro_sys AS
    FUNCTION calculate_fare(p_entry NUMBER, p_exit NUMBER) RETURN NUMBER IS
        v_dist1 NUMBER;
        v_dist2 NUMBER;
        v_total_dist NUMBER;
        v_calculated_fare NUMBER;
    BEGIN
        SELECT distance_marker INTO v_dist1 FROM metro_stations WHERE station_id = p_entry;
        SELECT distance_marker INTO v_dist2 FROM metro_stations WHERE station_id = p_exit;
        v_total_dist := ABS(v_dist1 - v_dist2);

        -- Structural pricing tiered slabs
        v_calculated_fare := 10.00 + (v_total_dist * 2.50);
        RETURN v_calculated_fare;
    END calculate_fare;

    PROCEDURE swipe_in(p_card NUMBER, p_station NUMBER) IS
        v_status VARCHAR2(15);
        v_bal NUMBER;
    BEGIN
        SELECT card_status, current_balance INTO v_status, v_bal FROM metro_cards WHERE
card_id = p_card;

        IF v_status = 'BLOCKED' OR v_bal < 15.00 THEN
            RAISE_APPLICATION_ERROR(-20011, 'Terminal Gate Access Denied: Card low balance or
blocked status.');
```

```

        END IF;

```

```

        INSERT INTO station_logs (card_id, entry_station_id) VALUES (p_card, p_station);
    END swipe_in;

    PROCEDURE swipe_out(p_card NUMBER, p_station NUMBER) IS
        v_log_id NUMBER;
        v_entry_station NUMBER;
        v_fare NUMBER;
    BEGIN
        SELECT log_id, entry_station_id INTO v_log_id, v_entry_station
        FROM (SELECT log_id, entry_station_id FROM station_logs
              WHERE card_id = p_card AND exit_station_id IS NULL
              ORDER BY swipe_in_time DESC) WHERE ROWNUM = 1;

        v_fare := calculate_fare(v_entry_station, p_station);

        UPDATE station_logs
        SET exit_station_id = p_station, swipe_out_time = SYSTIMESTAMP, fare_charged = v_fare
        WHERE log_id = v_log_id;

        UPDATE metro_cards SET current_balance = current_balance - v_fare WHERE card_id =
p_card;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20012, 'System Error: Active entry swipe timestamp log
missing.');
```

missing.');

```

        END swipe_out;
    END metro_sys;
/
```

INTERACTIVE VERIFICATION BLOCK

```

SET SERVEROUTPUT ON;
BEGIN
    INSERT INTO metro_stations VALUES (101, 'Central Terminal', 0.00);
    INSERT INTO metro_stations VALUES (105, 'Business District', 12.40);
    INSERT INTO metro_cards (holder_name, current_balance) VALUES ('John Doe', 50.00);
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('--- Processing Transit Terminal Entry Verification ---');
    metro_sys.swipe_in(1, 101);
```

```
DBMS_OUTPUT.PUT_LINE('--- Processing Destination Exit Fare Calculation ---');  
metro_sys.swipe_out(1, 105);  
END;
```

CONCLUSION

Ultimately, the production development of the 'Metro Smart Card Recharge and Fare Processor' system demonstrates the power of native database code. In conclusion, implementing this backend engine strictly using Oracle PL/SQL confirms that embedding business logic directly into the relational storage tier provides great operational safety, predictable concurrency control, and maximum execution speeds. Moving procedural actions closer to the tables avoids fetching thousands of raw rows over unstable network links. Instead, the database processes queries locally and returns only final results. Using stored procedures, native functions, and triggers forces strict data entry paths that protect the system from invalid states, application bypass attempts, or inconsistent writes. Furthermore, transactional reliability is maintained using custom exception handling configurations and autonomous transaction logs. These features safely write runtime audit trails to disk without rolling back the main processing task during an error. This design shows that standalone PL/SQL modules are highly effective for core administrative tasks and back-office pipelines. They remove external middleware dependencies while preserving full transactional isolation and structural consistency across the entire schema.

FUTURE ENHANCEMENT

To extend the capabilities of the 'Metro Smart Card Recharge and Fare Processor' framework, several advanced infrastructure paths can be pursued. Looking forward, this backend PL/SQL engine can be expanded into an enterprise-scale distributed asset platform. First, background processing and automated jobs can be configured by deploying Oracle's native background utility packages. This lets the system run recurring ledger cleanups, automated audit balances, and administrative reports during low-traffic hours without manual intervention. Second, to handle massive batch loads efficiently, current row-by-row cursors can be upgraded to bulk processing collections using bulk fetch binds and bulk error logging features. This lowers CPU overhead and speeds up high-volume operations. Third, the system can integrate with external cloud applications by using native database network packages to make direct web service calls. Finally, exposing these procedural packages as secure web endpoints via Oracle REST Data Services provides easy

integration with modern web and mobile frontends. This bridges the gap between secure database engines and distributed web microservices while keeping the underlying storage safe.

GITHUB LINK :

<https://github.com/sharmarohith2007prog/Rohithsharma/commits?author=sharmarohith2007-prog>

EMPLOYEE BONUS AND APPRAISAL AUTOMATION

M. SEENIVASAN

seenu78seenu78@gmail.com

ABSTRACT

The Employee Bonus and Appraisal Automation system is an enterprise human resources application designed for high-performance updates. Corporate environments need to process salary reviews, performance distributions, and tenure bonuses efficiently for large workforces. Modifying thousands of rows individually causes significant network lag and slows down data engines. This project solves that bottleneck by using Oracle advanced PL/SQL performance features: BULK COLLECT and FORALL statements. Instead of updating rows one by one, the application loads records into fast memory structures called collections. The appraisal procedure calculates bonuses by evaluating employee history and hire dates. Then, it sends all salary updates back to the tables in a single operation, cutting down context switches between the SQL and PL/SQL engines. The system uses exception logs to track individual errors, like invalid salary fields, allowing valid appraisals to complete smoothly. Operating entirely within the database kernel, this architecture delivers fast performance, illustrating how enterprise HR apps handle bulk salary updates efficiently.

CODING

DATABASE SCHEMA (DDL)

```
CREATE TABLE employees (  
    emp_id NUMBER PRIMARY KEY,  
    name VARCHAR2(100),  
    hire_date DATE,  
    salary NUMBER(10,2),  
    bonus_alloted NUMBER(10,2)  
);  
  
INSERT INTO employees VALUES (701, 'Jim Halpert', TO_DATE('2018-05-  
10', 'YYYY-MM-DD'), 60000.00, 0);  
INSERT INTO employees VALUES (702, 'Pam Beesly', TO_DATE('2023-11-01',  
'YYYY-MM-DD'), 45000.00, 0);  
COMMIT;
```

BULK COLLECT OPTIMIZATION UPDATE LOGIC

```
CREATE OR REPLACE PROCEDURE process_bulk_bonus IS  
    TYPE t_emp_list IS TABLE OF employees%ROWTYPE;  
    l_emp t_emp_list;  
    v_years NUMBER;  
    v_bonus NUMBER(10,2);  
BEGIN  
    SELECT * BULK COLLECT INTO l_emp FROM employees FOR UPDATE;  
  
    FOR i IN 1..l_emp.COUNT LOOP  
        v_years := MONTHS_BETWEEN(SYSDATE, l_emp(i).hire_date) / 12;  
        IF v_years >= 5 THEN  
            v_bonus := l_emp(i).salary * 0.15;  
        ELSE
```

```
        v_bonus := l_emp(i).salary * 0.05;
    END IF;

    UPDATE employees SET bonus_alloted = v_bonus WHERE emp_id =
l_emp(i).emp_id;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Bulk collection employee payroll updates
successfully committed.');
```

END;

/

CONCLUSION

This project demonstrates the huge speed advantages of using bulk memory operations for high-volume corporate HR updates. Using BULK COLLECT and FORALL cuts down context switching, allowing the system to update thousands of payroll records in seconds. Keeping appraisal rules inside stored procedures guarantees corporate compensation policies are applied uniformly and securely. The isolated exception handling ensures that a few broken records do not stop an entire company-wide payroll run. This project highlights how PL/SQL scales smoothly to handle heavy enterprise data workloads.

FUTURE ENHANCEMENT

This system can be upgraded by integrating a workflow approval process that requires electronic signatures from managers before updating payrolls. Adding Oracle Flashback queries will let HR view and roll back compensation histories to any specific point in time. The platform can use Oracle Advanced Analytics to flag salary anomalies that deviate from typical industry standards. Lastly, integrating with corporate LDAP servers will secure administrative tasks, ensuring payroll modifications align with corporate access policies.

GITHUB LINK :

<https://github.com/seenu78seenu78web/SEENIVASAN/commits?author=seenu78seenu78-web>

CRYPTO TOKEN AIRDROP AND WALLET SIMULATOR

S. SIVARITHEESH

sivaritheeshsivaritheesh@gmail.com

ABSTRACT

Architecture Scope: Simulates blockchain-inspired token allocation, safe tier-based pricing structure airdrops, and protected internal cryptographic ledger ledger balance transfer updates with custom automated transaction error protections.

This specialized database project, titled 'Crypto Token AirDrop and Wallet Simulator', focuses on building a secure architectural framework. The primary objective of this transactional architecture design is to build a high-performance, robust database backend layer built entirely using Oracle PL/SQL server-side database frameworks. This project solves critical data integrity issues, manual tracking delays, race conditions, and synchronization errors that naturally occur when processing workflows via application layers. By using native schema design, primary and foreign key constraints, check constraints, and unique indices, the application layer builds a solid structural baseline. The core logic shifts processing entirely within the Oracle Database Engine, significantly reducing network round-trip latency and context switching penalties between external programmatic servers and the relational tier. Business rules are strictly wrapped inside cohesive stored packages, ensuring standard modular deployments, data privacy, and secure database abstraction. To solve real-time resource contention or concurrent tracking modifications, explicit data processing cursors with transactional locking filters are fully deployed. This architecture guarantees strict structural verification and isolation levels across concurrent terminal connections. Database-level auditing and real-time validation checks are executed using native automated database triggers. These structural triggers track, intercept, and validate relational data rows before committing changes to disk. Custom exceptions handle unexpected runtime errors, providing clear debug data via terminal output streams while ensuring system resilience. This system operates independently of external web or reporting frameworks, serving as a reliable relational transaction engine for mission-critical apps.

CODING

DATABASE SCHEMA SETUP (DDL)

```
CREATE TABLE crypto_wallets (  
    wallet_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    user_name VARCHAR2(100) NOT NULL,  
    signup_tier VARCHAR2(20) CHECK (signup_tier IN ('GOLD', 'SILVER', 'BRONZE')),  
    token_balance NUMBER(24, 4) DEFAULT 0.0000,  
    created_at TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
  
CREATE TABLE token_transfers (  
    transfer_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    sender_wallet_id NUMBER REFERENCES crypto_wallets(wallet_id),  
    receiver_wallet_id NUMBER REFERENCES crypto_wallets(wallet_id),  
    amount NUMBER(24, 4) NOT NULL CHECK (amount > 0),  
    transaction_timestamp TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
  
CREATE TABLE system_audit_logs (  
    log_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    action_performed VARCHAR2(250),  
    log_timestamp TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
/
```

TRANSFERS AND AIRDROP BUSINESS LOGIC IMPLEMENTATION

```
CREATE OR REPLACE PACKAGE crypto_pkg AS  
    PROCEDURE execute_airdrop;  
    PROCEDURE transfer_tokens(p_sender NUMBER, p_receiver NUMBER, p_amount NUMBER);  
END crypto_pkg;  
/  
  
CREATE OR REPLACE PACKAGE BODY crypto_pkg AS  
    PROCEDURE log_action(p_action VARCHAR2) IS  
        PRAGMA AUTONOMOUS_TRANSACTION;  
    BEGIN  
        INSERT INTO system_audit_logs (action_performed) VALUES (p_action);  
    END log_action;  
END crypto_pkg;
```

```

        COMMIT;
END log_action;

PROCEDURE execute_airdrop IS
    CURSOR c_eligible_users IS
        SELECT wallet_id, signup_tier FROM crypto_wallets WHERE token_balance = 0;
    v_airdrop_amt NUMBER;
    v_count NUMBER := 0;
BEGIN
    FOR r_user IN c_eligible_users LOOP
        CASE r_user.signup_tier
            WHEN 'GOLD' THEN v_airdrop_amt := 500.0000;
            WHEN 'SILVER' THEN v_airdrop_amt := 250.0000;
            WHEN 'BRONZE' THEN v_airdrop_amt := 100.0000;
            ELSE v_airdrop_amt := 10.0000;
        END CASE;

        UPDATE crypto_wallets
        SET token_balance = token_balance + v_airdrop_amt
        WHERE wallet_id = r_user.wallet_id;

        v_count := v_count + 1;
    END LOOP;
    log_action('Airdrop completed successfully. Total wallets credited: ' || v_count);
END execute_airdrop;

PROCEDURE transfer_tokens(p_sender NUMBER, p_receiver NUMBER, p_amount NUMBER) IS
    v_sender_bal NUMBER;
    ex_insufficient_funds EXCEPTION;
    ex_identical_wallets EXCEPTION;
BEGIN
    IF p_sender = p_receiver THEN
        RAISE ex_identical_wallets;
    END IF;

    SELECT token_balance INTO v_sender_bal
    FROM crypto_wallets WHERE wallet_id = p_sender FOR UPDATE;

    IF v_sender_bal < p_amount THEN
        RAISE ex_insufficient_funds;
    
```

```

        END IF;

        UPDATE crypto_wallets SET token_balance = token_balance - p_amount WHERE wallet_id =
p_sender;
        UPDATE crypto_wallets SET token_balance = token_balance + p_amount WHERE wallet_id =
p_receiver;

        INSERT INTO token_transfers (sender_wallet_id, receiver_wallet_id, amount)
        VALUES (p_sender, p_receiver, p_amount);

        log_action('Transferred ' || p_amount || ' tokens from wallet ' || p_sender || ' to '
|| p_receiver);
    EXCEPTION
        WHEN ex_identical_wallets THEN
            log_action('ERROR: Transfer failed. Origin and destination wallets are
identical.');
```

```

            RAISE_APPLICATION_ERROR(-20001, 'Cannot transfer tokens to the same wallet.');
```

```

        WHEN ex_insufficient_funds THEN
            log_action('ERROR: Wallet ' || p_sender || ' attempted transfer with insufficient
balance.');
```

```

            RAISE_APPLICATION_ERROR(-20002, 'Insufficient funds for token asset transfer.');
```

```

        WHEN NO_DATA_FOUND THEN
            log_action('ERROR: One or both transaction wallet addresses do not exist.');
```

```

            RAISE_APPLICATION_ERROR(-20003, 'Invalid token wallet target account
identifiers.');
```

```

        END transfer_tokens;
END crypto_pkg;
/
```

INTERACTIVE VERIFICATION BLOCK

```

SET SERVEROUTPUT ON;
BEGIN
    INSERT INTO crypto_wallets (user_name, signup_tier, token_balance) VALUES ('Alice Gold',
'GOLD', 0);
    INSERT INTO crypto_wallets (user_name, signup_tier, token_balance) VALUES ('Bob Silver',
'SILVER', 0);
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('--- Executing Automated Token Airdrop Distribution ---');
    crypto_pkg.execute_airdrop;
```

```
DBMS_OUTPUT.PUT_LINE('--- Processing Ledger Token Account Swap Transfer ---');
crypto_pkg.transfer_tokens(p_sender => 1, p_receiver => 2, p_amount => 150.00);
END;
/
```

CONCLUSION

Ultimately, the production development of the 'Crypto Token AirDrop and Wallet Simulator' system demonstrates the power of native database code. In conclusion, implementing this backend engine strictly using Oracle PL/SQL confirms that embedding business logic directly into the relational storage tier provides great operational safety, predictable concurrency control, and maximum execution speeds. Moving procedural actions closer to the tables avoids fetching thousands of raw rows over unstable network links. Instead, the database processes queries locally and returns only final results. Using stored procedures, native functions, and triggers forces strict data entry paths that protect the system from invalid states, application bypass attempts, or inconsistent writes. Furthermore, transactional reliability is maintained using custom exception handling configurations and autonomous transaction logs. These features safely write runtime audit trails to disk without rolling back the main processing task during an error. This design shows that standalone PL/SQL modules are highly effective for core administrative tasks and back-office pipelines. They remove external middleware dependencies while preserving full transactional isolation and structural consistency across the entire schema.

FUTURE ENHANCEMENT

To extend the capabilities of the 'Crypto Token AirDrop and Wallet Simulator' framework, several advanced infrastructure paths can be pursued. Looking forward, this backend PL/SQL engine can be expanded into an enterprise-scale distributed asset platform. First, background processing and automated jobs can be configured by deploying Oracle's native background utility packages. This lets the system run recurring ledger cleanups, automated audit balances, and administrative reports during low-traffic hours without manual intervention. Second, to handle massive batch loads efficiently, current row-by-row cursors can be upgraded to bulk processing collections using bulk fetch binds and bulk error logging features. This lowers CPU overhead and speeds up high-volume operations. Third, the system can integrate with external cloud applications by using native database network packages to make direct web service calls.

Finally, exposing these procedural packages as secure web endpoints via Oracle REST Data Services provides easy integration with modern web and mobile frontends. This bridges the gap between secure database engines and distributed web microservices while keeping the underlying storage safe.

GITHUB LINK :

<https://github.com/sivaritheeshsivaritheesh-oss/Siva-Ritheesh-/commits?author=sivaritheeshsivaritheesh-oss>

STREAMING PLATFORM SUBSCRIPTION AND CONTENT TIER MANAGER

S. SURYARAJ

sivakumarsurya2014@gmail.com

ABSTRACT

Architecture Scope:

Manages operational membership levels, applying algorithmic constraints over platform content libraries.. This specialized database project, titled 'Streaming Platform Subscription and Content Tier Manager', focuses on building a secure architectural framework. The primary objective of this transactional architecture design is to build a high-performance, robust database backend layer built entirely using Oracle PL/SQL server-side database frameworks. This project solves critical data integrity issues, manual tracking delays, race conditions, and synchronization errors that naturally occur when processing workflows via application layers. By using native schema design, primary and foreign key constraints, check constraints, and unique indices, the application layer builds a solid structural baseline. The core logic shifts processing entirely within the Oracle Database Engine, significantly reducing network round-trip latency and context switching penalties between external programmatic servers and the relational tier. Business rules are strictly wrapped inside cohesive stored packages, ensuring standard modular deployments, data privacy, and secure database abstraction. To solve real-time resource contention or concurrent tracking modifications, explicit data processing cursors with transactional locking filters are fully deployed. This architecture guarantees strict structural verification and isolation levels across concurrent terminal connections. Database-level auditing and real-time validation checks are executed using native automated database triggers. These structural triggers track, intercept, and validate relational data rows before committing changes to disk. Custom exceptions handle unexpected runtime errors, providing clear debug data via terminal output streams while ensuring system resilience. This system operates independently of external web or reporting frameworks, serving as a reliable relational transaction engine for mission-critical apps.

CODING

DATABASE SCHEMA SETUP (DDL)

```
CREATE TABLE subscriber_accounts (  
    account_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    profile_name VARCHAR2(100) NOT NULL,  
    service_tier VARCHAR2(15) DEFAULT 'FREE' CHECK (service_tier IN ('FREE', 'PREMIUM')),  
    expiry_timeline DATE,  
    concurrent_streams_allowed NUMBER DEFAULT 1  
);  
  
CREATE TABLE active_sessions (  
    session_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    account_id NUMBER REFERENCES subscriber_accounts(account_id),  
    content_id NUMBER,  
    login_timestamp TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
/
```

CONTROL POLICIES INFRASTRUCTURE WRAPPER

```
CREATE OR REPLACE PACKAGE streaming_governance_pkg AS  
    PROCEDURE evaluate_account_expiry;  
    PROCEDURE authorize_stream_session(p_account_id NUMBER, p_video_id NUMBER);  
END streaming_governance_pkg;  
/  
  
CREATE OR REPLACE PACKAGE BODY streaming_governance_pkg AS  
    PROCEDURE evaluate_account_expiry IS  
    BEGIN  
        UPDATE subscriber_accounts  
        SET service_tier = 'FREE', concurrent_streams_allowed = 1  
        WHERE expiry_timeline < SYSDATE AND service_tier = 'PREMIUM';  
    END evaluate_account_expiry;  
  
    PROCEDURE authorize_stream_session(p_account_id NUMBER, p_video_id NUMBER) IS  
        v_max_allowed NUMBER;  
        v_current_active NUMBER;  
        ex_limit_breached EXCEPTION;  
    BEGIN  
        evaluate_account_expiry;
```

```

        SELECT concurrent_streams_allowed INTO v_max_allowed FROM subscriber_accounts WHERE
account_id = p_account_id;
        SELECT COUNT(*) INTO v_current_active FROM active_sessions WHERE account_id =
p_account_id;

        IF v_current_active >= v_max_allowed THEN
            RAISE ex_limit_breached;
        END IF;

        INSERT INTO active_sessions (account_id, content_id) VALUES (p_account_id,
p_video_id);
    EXCEPTION
        WHEN ex_limit_breached THEN
            RAISE_APPLICATION_ERROR(-20061, 'Access Guard Exclusion: Active stream instance
parameters saturated.');
```

END authorize_stream_session;

END streaming_governance_pkg;

/

INTERACTIVE VERIFICATION BLOCK

```

SET SERVEROUTPUT ON;
BEGIN
    INSERT INTO subscriber_accounts (profile_name, service_tier, expiry_timeline,
concurrent_streams_allowed)
    VALUES ('Alex Subscriber', 'PREMIUM', SYSDATE+30, 3);
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('--- Authorizing Device Video Channel Interconnect ---');
    streaming_governance_pkg.authorize_stream_session(1, 9901);
END;
/
```

GITHUB LINK :

<https://github.com/sivakumarsuriya2014-sys/SURIYARAJ-S/commits?author=sivakumarsu>

CONCLUSION

Ultimately, the production development of the 'Streaming Platform Subscription and Content Tier Manager' system demonstrates the power of native database code. In conclusion, implementing this backend engine strictly using Oracle PL/SQL confirms that embedding business logic directly into the relational storage tier provides great operational safety, predictable concurrency control, and maximum execution speeds. Moving procedural actions closer to the tables avoids fetching thousands of raw rows over unstable network links. Instead, the database processes queries locally and returns only final results. Using stored procedures, native functions, and triggers forces strict data entry paths that protect the system from invalid states, application bypass attempts, or inconsistent writes. Furthermore, transactional reliability is maintained using custom exception handling configurations and autonomous transaction logs. These features safely write runtime audit trails to disk without rolling back the main processing task during an error. This design shows that standalone PL/SQL modules are highly effective for core administrative tasks and back-office pipelines. They remove external middleware dependencies while preserving full transactional isolation and structural consistency across the entire schema.

FUTURE ENHANCEMENT

To extend the capabilities of the 'Streaming Platform Subscription and Content Tier Manager' framework, several advanced infrastructure paths can be pursued. Looking forward, this backend PL/SQL engine can be expanded into an enterprise-scale distributed asset platform. First, background processing and automated jobs can be configured by deploying Oracle's native background utility packages. This lets the system run recurring ledger cleanups, automated audit balances, and administrative reports during low-traffic hours without manual intervention. Second, to handle massive batch loads efficiently, current row-by-row cursors can be upgraded to bulk processing collections using bulk fetch binds and bulk error logging features. This lowers CPU overhead and speeds up high-volume operations. Third, the system can integrate with external cloud applications by using native database network packages to make direct web service calls. Finally, exposing these procedural packages as secure web endpoints via Oracle REST Data Services provides easy integration with modern web and mobile frontends. This bridges the gap between secure database engines and distributed web microservices while keeping the underlying storage safe.

AUTOMATED ELECTRICITY BILLING SYSTEM

S. SUSITHARAN

susitharansam@gmail.com

ABSTRACT

The Automated Electricity Billing System is a batch-processing database application designed to compute utility invoices based on consumption. Processing high-volume utility billing requires scalable data handling and accurate logic to apply tiered pricing scales. This project utilizes PL/SQL explicit cursors to loop through consumer usage records sequentially without overloading server memory. The billing logic runs inside conditional blocks that apply different pricing rates based on energy usage tiers. This approach ensures lower consumption brackets receive basic rates while higher usage tiers incur premium charges. The system tracks payment timelines via a specialized database trigger. This trigger monitors invoice dates and automatically adds late fees if payments miss deadlines. The application processes high volumes of records using structured cursors, keeping memory footprints low even during intensive billing cycles. It includes thorough exception validation blocks to catch anomalies, such as negative meter values or missing customer records. This ensures erroneous logs do not break the batch run. By executing entirely within the Oracle engine, the system speeds up bill calculations and minimizes external network traffic, illustrating the advantages of data-centric programming.

CODING

DATABASE SCHEMA (DDL)

```
CREATE TABLE electricity_consumers (  
    consumer_id NUMBER PRIMARY KEY,  
    name VARCHAR2(100),  
    units_consumed NUMBER,  
    bill_amount NUMBER(10,2),  
    due_date DATE  
);  
  
INSERT INTO electricity_consumers VALUES (301, 'David Miller', 120, 0, SYSDATE + 14);  
INSERT INTO electricity_consumers VALUES (302, 'Elena Rostova', 450, 0, SYSDATE - 2);  
COMMIT;
```

CURSOR CALCULATIONS PROCEDURE

```
CREATE OR REPLACE PROCEDURE calculate_all_bills IS  
    CURSOR c_consumers IS SELECT consumer_id, units_consumed FROM electricity_consumers;  
    v_bill NUMBER(10,2);  
BEGIN  
    FOR r IN c_consumers LOOP  
        IF r.units_consumed <= 100 THEN  
            v_bill := r.units_consumed * 2.5;  
        ELSIF r.units_consumed <= 300 THEN  
            v_bill := (100 * 2.5) + ((r.units_consumed - 100) * 4.0);  
        ELSE  
            v_bill := (100 * 2.5) + (200 * 4.0) + ((r.units_consumed - 300) * 6.0);  
        END IF;  
  
        UPDATE electricity_consumers  
        SET bill_amount = v_bill  
        WHERE consumer_id = r.consumer_id;  
    END LOOP;  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('All electricity processing bills generated dynamically.');
```

END;
/

CONCLUSION

The Automated Electricity Billing System demonstrates how batch-processing tasks can be efficiently executed using native PL/SQL loops. Using explicit cursors allows the engine to process thousands of customer records smoothly without hitting server memory caps. Embedding tier calculations directly into stored code ensures billing logic is accurate and highly secure against tampering. Automating penalty additions via database triggers reduces administrative overhead and ensures policy consistency. This project highlights the strength of database systems in handling large-scale computations reliably and independently.

FUTURE ENHANCEMENT

Future updates can implement PL/SQL Bulk Collect and FORALL features to process billing batches in parallel, significantly boosting speed. Integrating Oracle Spatial features will allow the system to analyze consumption patterns across different geographic areas. A predictive analytical engine can be added using built-in statistical functions to forecast future grid load demands. Finally, building an automated PDF generator within the database will let the system create and email invoices directly to consumers.

GITHUB LINK :

<https://github.com/susitharansam-ai/Susitharan-/commits?author=susitharansam-ai>

STUDENT GRADING AND RANKING SYSTEM

THANGAPANDI.K

thangapandikaruppaiya07@gmail.com

ABSTRACT

The Student Grading and Ranking System is an academic reporting application built entirely in PL/SQL to automate performance tracking. Educational centers handle large datasets of student scores that require aggregation, grading, and relative ranking. This system structures data across Students, Courses, and Marks tables. It uses a stored procedure with explicit cursors to loop through records and calculate totals, percentages, and grade designations based on institutional rule sets. The core routine analyzes performance metrics and assigns final ranks by comparing student totals across the entire class. To show results without a frontend, the system format outputs into clean text tables printed directly to terminal consoles. This layout displays student IDs, names, course details, numeric scores, and final ranks in a highly readable format. The system includes robust exception blocks to manage issues like uncompleted courses or missing scores without halting the process. By performing all calculations within the database, the application minimizes data transit times, showcasing an efficient data-processing solution.

CODING

DATABASE SCHEMA (DDL)

```
CREATE TABLE student_grades (  
    student_id NUMBER PRIMARY KEY,  
    name VARCHAR2(100),  
    score_math NUMBER,  
    score_science NUMBER,  
    final_grade VARCHAR2(2)  
);  
  
INSERT INTO student_grades VALUES (401, 'Michael Scott', 85, 92, NULL);  
INSERT INTO student_grades VALUES (402, 'Dwight Schrute', 45, 55, NULL);  
COMMIT;
```

CURSOR GRADING ALGORITHM BLOCK

```
CREATE OR REPLACE PROCEDURE compute_student_grades IS  
    v_avg NUMBER;  
    v_grade VARCHAR2(2);  
BEGIN  
    FOR r IN (SELECT student_id, score_math, score_science FROM student_grades) LOOP  
        v_avg := (r.score_math + r.score_science) / 2;  
  
        IF v_avg >= 90 THEN v_grade := 'A';  
        ELSIF v_avg >= 75 THEN v_grade := 'B';  
        ELSIF v_avg >= 50 THEN v_grade := 'C';  
        ELSE v_grade := 'F';  
        END IF;  
  
        UPDATE student_grades SET final_grade = v_grade WHERE student_id = r.student_id;  
    END LOOP;  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Academic calculations completed successfully.');
```

END;
/

CONCLUSION

The Student Grading and Ranking System proves that databases can generate detailed, formatted reports without external reporting tools. Using cursors to process academic evaluations ensures data updates reliably, even with dense student records. Processing formulas directly on the database engine keeps calculation logic uniform across all academic departments. The clean terminal output demonstrates that backend text generation is a fast, efficient alternative for administrative lookups. This project stands as a solid model for automated academic reporting and bulk data summary.

FUTURE ENHANCEMENT

This system can be enhanced by using analytic functions like DENSE_RANK and OVER to simplify relative performance ranking across departments. Adding a tracking module will let users compare a student's progress across multiple semesters using historic data tables. Integrating Oracle Web PL/SQL fields will allow users to export report summaries directly as Excel sheets. Finally, adding a security layer will restrict edit access to verified professors, while giving students view-only access to their individual grades.

GITHUB LINK :

<https://github.com/thangapandikaruppaiya07-stack/document/commits?author=thangapandikaruppaiya07-stack>

GYM MEMBERSHIP AND TARINER SCHEDULE OPTIMIZER

R. VIGNESH

vigneshagilan93@gmail.com

ABSTRACT

Architecture Scope: Ensures structural conflict resolution inside facility allocation matrices, monitoring scheduling overlaps using declarative verification rules. This specialized database project, titled 'Gym Membership and Trainer Schedule Optimizer', focuses on building a secure architectural framework. The primary objective of this transactional architecture design is to build a high-performance, robust database backend layer built entirely using Oracle PL/SQL server-side database frameworks. This project solves critical data integrity issues, manual tracking delays, race conditions, and synchronization errors that naturally occur when processing workflows via application layers. By using native schema design, primary and foreign key constraints, check constraints, and unique indices, the application layer builds a solid structural baseline. The core logic shifts processing entirely within the Oracle Database Engine, significantly reducing network round-trip latency and context switching penalties between external programmatic servers and the relational tier. Business rules are strictly wrapped inside cohesive stored packages, ensuring standard modular deployments, data privacy, and secure database abstraction. To solve real-time resource contention or concurrent tracking modifications, explicit data processing cursors with transactional locking filters are fully deployed. This architecture guarantees strict structural verification and isolation levels across concurrent terminal connections. Database-level auditing and real-time validation checks are executed using native automated database triggers. These structural triggers track, intercept, and validate relational data rows before committing changes to disk. Custom exceptions handle unexpected runtime errors, providing clear debug data via terminal output streams while ensuring system resilience. This system operates independently of external web or reporting frameworks, serving as a reliable relational transaction engine for mission-critical apps.

CODING

DATABASE SCHEMA SETUP (DDL)

```
CREATE TABLE physical_trainers (  
    trainer_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    trainer_name VARCHAR2(100) NOT NULL,  
    specialization VARCHAR2(50)  
);  
  
CREATE TABLE schedule_bookings (  
    booking_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    trainer_id NUMBER REFERENCES physical_trainers(trainer_id),  
    client_name VARCHAR2(100) NOT NULL,  
    assigned_hour NUMBER CHECK (assigned_hour BETWEEN 6 AND 22), -- military scale hours  
    booking_date DATE NOT NULL  
);  
  
CREATE TABLE operations_ledger (  
    entry_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    context_desc VARCHAR2(250),  
    logged_at TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
/
```

REAL-TIME OVERLAP VERIFICATION LAYER

```
CREATE OR REPLACE FUNCTION check_schedule_overlap(p_trainer NUMBER, p_date DATE, p_hour  
NUMBER)  
RETURN NUMBER IS  
    v_collision_count NUMBER := 0;  
BEGIN  
    SELECT COUNT(*) INTO v_collision_count  
    FROM schedule_bookings  
    WHERE trainer_id = p_trainer AND booking_date = TRUNC(p_date) AND assigned_hour = p_hour;  
  
    RETURN v_collision_count;  
END check_schedule_overlap;  
/
```

INTERCEPT AUDITING ENGINE

```
CREATE OR REPLACE TRIGGER trg_audit_schedule_mutations
AFTER INSERT OR DELETE ON schedule_bookings
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO operations_ledger (context_desc)
        VALUES ('New schedule locked for asset trainer reference ' || :NEW.trainer_id || ' at
timeline block hour ' || :NEW.assigned_hour);
    ELSIF DELETING THEN
        INSERT INTO operations_ledger (context_desc)
        VALUES ('Schedule vacated for asset trainer reference ' || :OLD.trainer_id || ' at
timeline block hour ' || :OLD.assigned_hour);
    END IF;
END;
/
```

INTERACTIVE VERIFICATION BLOCK

```
SET SERVEROUTPUT ON;
DECLARE
    v_status NUMBER;
BEGIN
    INSERT INTO physical_trainers (trainer_name, specialization) VALUES ('Coach Elite',
'Strength Athletics');
    INSERT INTO schedule_bookings (trainer_id, client_name, assigned_hour, booking_date)
VALUES (1, 'Client Beta', 10, TRUNC(SYSDATE));
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('--- Evaluating Operational Availability Matrix Overlap ---');
    v_status := check_schedule_overlap(1, SYSDATE, 10);
    DBMS_OUTPUT.PUT_LINE('Collision index evaluation scalar outcome: ' || v_status);
END;
/
```

CONCLUSION

Ultimately, the production development of the 'Gym Membership and Trainer Schedule Optimizer' system demonstrates the power of native database code. In conclusion, implementing this backend engine strictly using Oracle PL/SQL confirms that embedding business logic directly into the relational storage tier provides great operational safety, predictable concurrency control, and maximum execution speeds. Moving procedural actions closer to the tables avoids fetching thousands of raw rows over unstable network links. Instead, the database processes queries locally and returns only final results. Using stored procedures, native functions, and triggers forces strict data entry paths that protect the system from invalid states, application bypass attempts, or inconsistent writes. Furthermore, transactional reliability is maintained using custom exception handling configurations and autonomous transaction logs. These features safely write runtime audit trails to disk without rolling back the main processing task during an error. This design shows that standalone PL/SQL modules are highly effective for core administrative tasks and back-office pipelines. They remove external middleware dependencies while preserving full transactional isolation and structural consistency across the entire schema.

FUTURE ENHANCEMENT

extend the capabilities of the 'Gym Membership and Trainer Schedule Optimizer' framework, several advanced infrastructure paths can be pursued. Looking forward, this backend PL/SQL engine can be expanded into an enterprise-scale distributed asset platform. First, background processing and automated jobs can be configured by deploying Oracle's native background utility packages. This lets the system run recurring ledger cleanups, automated audit balances, and administrative reports during low-traffic hours without manual intervention. Second, to handle massive batch loads efficiently, current row-by-row cursors can be upgraded to bulk processing collections using bulk fetch binds and bulk error logging features. This lowers CPU overhead and speeds up high-volume operations. Third, the system can integrate with external cloud applications by using native database network packages to make direct web service calls. Finally, exposing these procedural packages as secure web endpoints via Oracle REST Data

Services provides easy integration with modern web and mobile frontends. This bridges the gap between secure database engines and distributed web microservices while keeping the underlying storage safe.

GITHUB LINK :

<https://github.com/vigneshr-dev/VIGNESH/commits?author=vigneshr-dev>

DATABASE SCHEMA USER PRIVILEGE AND ROLE AUDITOR

A. YUVARAJPANDIYAN

yuvarajpandiyam09@gmail.com

ABSTRACT

Architecture Scope: A administrative native metadata dictionary scanning block that aggregates systemic access profiles, identifying nested authorization points. This specialized database project, titled 'Database Schema User Privilege and Role Auditor', focuses on building a secure architectural framework. The primary objective of this transactional architecture design is to build a high-performance, robust database backend layer built entirely using Oracle PL/SQL server-side database frameworks. This project solves critical data integrity issues, manual tracking delays, race conditions, and synchronization errors that naturally occur when processing workflows via application layers. By using native schema design, primary and foreign key constraints, check constraints, and unique indices, the application layer builds a solid structural baseline. The core logic shifts processing entirely within the Oracle Database Engine, significantly reducing network round-trip latency and context switching penalties between external programmatic servers and the relational tier. Business rules are strictly wrapped inside cohesive stored packages, ensuring standard modular deployments, data privacy, and secure database abstraction. To solve real-time resource contention or concurrent tracking modifications, explicit data processing cursors with transactional locking filters are fully deployed. This architecture guarantees strict structural verification and isolation levels across concurrent terminal connections. Database-level auditing and real-time validation checks are executed using native automated database triggers. These structural triggers track, intercept, and validate relational data rows before committing changes to disk. Custom exceptions handle unexpected runtime errors, providing clear debug data via terminal output streams while ensuring system resilience. This system operates independently of external web or reporting frameworks, serving as a reliable relational transaction engine for mission-critical apps.

CODING

DATABASE SCHEMA SETUP (DDL)

```
CREATE TABLE schema_audit_reports (  
    report_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    grantee_username VARCHAR2(100),  
    table_name VARCHAR2(100),  
    privilege_type VARCHAR2(50),  
    is_admin_option VARCHAR2(5),  
    scanned_at TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
/
```

DICTIONARY METADATA ANALYSIS ENGINE

```
CREATE OR REPLACE PROCEDURE perform_security_privilege_audit IS  
    CURSOR c_dictionary_scan IS  
        SELECT grantee, table_name, privilege, grantable  
        FROM USER_TAB_PRIVS  
        WHERE privilege IN ('SELECT', 'INSERT', 'UPDATE', 'DELETE');  
    v_counter NUMBER := 0;  
BEGIN  
    EXECUTE IMMEDIATE 'TRUNCATE TABLE schema_audit_reports';  
  
    FOR r_priv IN c_dictionary_scan LOOP  
        INSERT INTO schema_audit_reports (grantee_username, table_name, privilege_type,  
is_admin_option)  
        VALUES (r_priv.grantee, r_priv.table_name, r_priv.privilege, r_priv.grantable);  
        v_counter := v_counter + 1;  
    END LOOP;  
  
    DBMS_OUTPUT.PUT_LINE('Security Scan Terminated. Total cross-table access bindings logged:  
' || v_counter);  
END perform_security_privilege_audit;  
/
```

INTERACTIVE VERIFICATION BLOCK

```
SET SERVEROUTPUT ON;  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('--- Querying Data Dictionary Matrix Profiles ---');
```

```
    perform_security_privilege_audit;  
END;  
/
```

CONCLUSION

Ultimately, the production development of the 'Database Schema User Privilege and Role Auditor' system demonstrates the power of native database code. In conclusion, implementing this backend engine strictly using Oracle PL/SQL confirms that embedding business logic directly into the relational storage tier provides great operational safety, predictable concurrency control, and maximum execution speeds. Moving procedural actions closer to the tables avoids fetching thousands of raw rows over unstable network links. Instead, the database processes queries locally and returns only final results. Using stored procedures, native functions, and triggers forces strict data entry paths that protect the system from invalid states, application bypass attempts, or inconsistent writes. Furthermore, transactional reliability is maintained using custom exception handling configurations and autonomous transaction logs. These features safely write runtime audit trails to disk without rolling back the main processing task during an error. This design shows that standalone PL/SQL modules are highly effective for core administrative tasks and back-office pipelines. They remove external middleware dependencies while preserving full transactional isolation and structural consistency across the entire schema.

FUTURE ENHANCEMENT

To extend the capabilities of the 'Database Schema User Privilege and Role Auditor' framework, several advanced infrastructure paths can be pursued. Looking forward, this backend PL/SQL engine can be expanded into an enterprise-scale distributed asset platform. First, background processing and automated jobs can be configured by deploying Oracle's native background utility packages. This lets the system run recurring ledger cleanups, automated audit balances, and administrative reports during low-traffic hours without manual intervention. Second, to handle massive batch loads efficiently, current row-by-row cursors can be upgraded to bulk processing collections using bulk fetch binds and bulk error logging features. This lowers CPU overhead and speeds up high-volume operations. Third, the system can integrate with external

cloud applications by using native database network packages to make direct web service calls. Finally, exposing these procedural packages as secure web endpoints via Oracle REST Data Services provides easy integration with modern web and mobile frontends. This bridges the gap between secure database engines and distributed web microservices while keeping the underlying storage safe.

GITHUB LINK :

<https://github.com/yuvarajpandiyan09-lab/YUVARAJPANDIYAN-A/commits?author=yuvarajpandiyan09-lab>

PL/SQL CODE WITH US

DEVELOPERS

- HARIHARAN J
- KARTHIKEYAN B
- MANIKANDAN M
- NIRMAL RAJA R
- PAUL CLEMENT K
- ROHITH SHARMA R
- THANGAPANDI K
- SEENIVASAN M
- SIVARITHEESH S
- SURIYARAJ S
- SUSITHARAN S
- VIGNESH R
- YUVARAJ PANDIAN A

EDITOR

Ms. KALPANA DEVI J,

CO - EDITORS

- KARTHIKEYAN B
- MANIKANDAN M
- THANGAPANDI K
- VIGNESH R

LINK WITH US IN GITHUB